

PROMPT CODEX — Corrections Titanium v12

Contexte

Tu intervies sur **Titanium v12**, un bot de trading algorithmique Python tournant sur Windows. Le projet est dans C:\Users\florence\Desktop\v12 (65 073 fichiers, 4,67 Go). Le backend FastAPI tourne sur localhost:8090. Le MCP Server communique via stdio avec Antigravity Cascade.

Objectif

Appliquer 7 corrections critiques sur 2 fichiers uniquement. **Ne toucher à aucun autre fichier.** Garder la signature publique (noms de fonctions, paramètres, types de retour) inchangée pour ne pas casser les appelants.

Fichier 1 : data/mt5_provider.py

Correction A — Verrouiller `_selected` (CRITIQUE)

Problème : Le set `_selected` est lu/écrit hors du `mt5_lock` dans `get_rates()`, `get_ohlcv()`, `get_tick()`, `get_ticks_fast()`. En concurrence (forex + swing + scan), deux threads appellent `mt5.symbol_select()` simultanément sur le même symbole → MT5 retourne `None` ou plante.

Action :

1. Créer un verrou dédié `_selected_lock = threading.Lock()`
2. Wrapper TOUT accès à `_selected` (lecture ET écriture) avec `_selected_lock`
3. Vérifier le booléen retourné par `mt5.symbol_select(symbol, True)`. Si `False`, lever une exception `MT5SymbolError` avec le message d'erreur MT5.
4. Extraire la logique de sélection dans une fonction privée `_ensure_symbol(symbol: str) -> None`

Code attendu (extrait) :

```
_selected: set = set()
_selected_lock = threading.Lock()

class MT5SymbolError(Exception): pass

def _ensure_symbol(symbol: str) -> None:
    with _selected_lock:
        if symbol in _selected:
            return
        with mt5_lock:
            ok = mt5.symbol_select(symbol, True)
            if not ok:
                err = mt5.last_error()
                raise MT5SymbolError(f"symbol_select({symbol}) échoué: {err}")
        _selected.add(symbol)
```

Correction B — Retry backoff sur `ensure_init()` (HAUTE)

Problème : `mt5.initialize()` est appelé une seule fois. Si MT5 est fermé ou en cours de démarrage, le bot reste dans un état `_initialized = False` permanent jusqu'au redémarrage manuel.

Action :

1. Ajouter `_init_failures` (compteur) et `_init_last_try` (timestamp) globaux
2. Implémenter un backoff exponentiel : attendre `min(backoff * 2^n, 30s)` entre les tentatives
3. `max_retries=3` par défaut
4. Vérification proactive : si `_initialized` est `True` mais `mt5.terminal_info()` retourne `None`, forcer `_initialized = False` et retenter

Code attendu (extrait) :

```
_init_failures = 0
_init_last_try = 0.0

def ensure_init(max_retries: int = 3, backoff: float = 1.0) -> bool:
    global _initialized, _init_failures, _init_last_try
    if not MT5_AVAILABLE:
        raise MT5DisconnectedError("MetaTrader5 non installé")
    if _initialized:
        try:
            if mt5.terminal_info() is None:
                _initialized = False
        except Exception:
            _initialized = False
    if _initialized:
        return True
    now = time.monotonic()
    if now - _init_last_try < min(backoff * (2 ** _init_failures), 30.0):
        return False
    _init_last_try = now
    with mt5_lock:
        if _initialized: return True
        for attempt in range(1, max_retries + 1):
            if mt5.initialize():
                _initialized = True; _init_failures = 0; return True
            if attempt < max_retries:
                time.sleep(backoff * (2 ** (attempt - 1)))
        _init_failures += 1
        return False
```

Correction C — Exceptions explicites au lieu de `None` (HAUTE)

Problème : `get_rates()`, `get_ohlcv()`, `get_tick()` retournent `None` quand les données sont indisponibles. L'appelant ne peut pas distinguer “MT5 déconnecté”, “symbole invalide” et “marché fermé”.

Action :

1. Créer 3 exceptions : `MT5DisconnectedError`, `MT5DataError`, `MT5SymbolError`

2. Remplacer TOUS les `return None` par `raise ExceptionAppropriée("message détaillé")`
3. `get_ticks_fast()` doit capturer les erreurs par symbole et retourner uniquement les symboles valides (ne pas planter le batch entier)

Règle : Les fonctions publiques retournent désormais `pd.DataFrame` ou `dict` (jamais `None`). L'appelant utilise `try/except`.

Fichier 2 : `mcp_server.py`

Correction D — Client HTTP persistant (HAUTE)

Problème : Chaque requête crée `httpx.AsyncClient(timeout=_HTTP_TIMEOUT)` puis le jette. Pas de keep-alive TCP, latence +200ms par requête, épuisement des ports si charge élevée.

Action :

1. Créer un client global `_http_client: httpx.AsyncClient | None = None`
2. Fonction `_get_client()` qui retourne le client existant ou en crée un avec `limits=httpx.Limits(max_connections=20, max_keepalive_connections=10)`
3. Fonction `_close_client()` pour fermeture propre au shutdown
4. `_get()` et `_post()` utilisent `_get_client()` au lieu de `async with httpx.AsyncClient(...)`

Correction E — Timeout 10s + Retry réseau (HAUTE)

Problème : `_HTTP_TIMEOUT = 4.0` est trop court quand Titanium charge un modèle `.pk1` (925 Mo). Aucun retry sur erreur 5xx ou timeout → faux positif `"status": "offline"`.

Action :

1. `_HTTP_TIMEOUT = 10.0`
2. `_HTTP_MAX_RETRIES = 2`
3. `_HTTP_RETRY_BACKOFF = 1.0`
4. Dans `_get()` et `_post()` : boucle `for attempt in range(1, _HTTP_MAX_RETRIES + 1)`
5. Distinction des erreurs :
 - `httpx.TimeoutException` → `"status": "timeout"` (retryable)
 - `httpx.ConnectError` → `"status": "offline"` (retryable)
 - HTTP 5xx → retry avec backoff
 - HTTP 4xx → `"status": "error"` (non retryable)

Correction F — Imports dynamiques mis en cache (MOYEN)

Problème : `modify_scoring_weight` et `modify_risk_params` font `import sys, os + sys.path.insert + from engine.learning_engine import ...` à chaque appel de tool (100-500ms de latence).

Action :

1. Créer un cache global `_import_cache: dict[str, Any] = {}`

2. Fonction `_cached_import(module_path: str)` qui importe une seule fois
3. Remplacer tous les imports dynamiques dans `call_tool()` par `_cached_import("engine.learning_engine")`, etc.

Correction G — Health check proactif au démarrage (MOYEN)

Problème : Le serveur MCP démarre même si Titanium est down. Les premières requêtes échouent silencieusement.

Action :

1. Fonction `_health_check()` -> dict qui appelle `await _get("/api/state")`
 2. Dans `_serve()` : appeler `_health_check()` avant `async with stdio_server()`
 3. Loguer sur stderr : `□ Connecté à Titanium vX` ou `□ Titanium backend non accessible`
-

Contraintes absolues

- **Ne PAS modifier** les signatures des fonctions publiques (`ensure_init`, `get_rates`, `get_ohlcv`, `get_tick`, `get_ticks_fast`, `account_snapshot`, `read_resource`, `call_tool`, `list_resources`, `list_tools`)
- **Ne PAS modifier** `test_mcp_server.py` — les tests doivent toujours passer (mock `_get`)
- **Ne PAS modifier** les URI des ressources MCP
- **Ne PAS modifier** les noms des tools MCP
- Garder le docstring et les commentaires en français
- Windows uniquement (pas de code Unix-specific)
- Python 3.11+

Livrables attendus

1. `data/mt5_provider.py` corrigé
2. `mcp_server.py` corrigé
3. Un résumé des changements ligne par ligne

Références

Utilise les fichiers de patch fournis comme référence exacte de la structure attendue, mais adapte-les pour qu'ils s'intègrent proprement au codebase existant (notamment les imports relatifs et les noms de modules).